

OpenStack RCA Copilot

Root Cause Analysis for Cloud Infrastructure

Design, Evidence Pipeline and Development.

HANFAOUI Karim¹

¹EMSI, 4th Year IA & DATA – Summer Internship @ Huawei
Supervisor: HINGER Anass
Karim.Hanfaoui@emsi-edu.ma

Huawei , Casablanca, July 2026

Table of Contents

Role: Present the full report structure and the order of the steps

Project Stage: Navigation / global framing

- 1 Problem Statement and Methodology
- 2 OpenStack and the Debugging Challenge
- 3 Data Ingestion and Structuring Pipeline
- 4 Correlation Graph and Evidence Package
- 5 AI Reasoning Layer
- 6 Data Sovereignty and Roadmap

List of Figures I

Role: List all diagrams and figures presented in the report

Project Stage: Navigation / global framing

Figure 1. PERT network of the OpenStack RCA Copilot with critical path and earliest/latest dates.	10
Figure 2. GitHub repository QR code	20
Figure 3. Official benchmark comparison	31

Table of Contents

- 1 Problem Statement and Methodology
- 2 OpenStack and the Debugging Challenge
- 3 Data Ingestion and Structuring Pipeline
- 4 Correlation Graph and Evidence Package
- 5 AI Reasoning Layer
- 6 Data Sovereignty and Roadmap

Problem Statement

Role: State the operational need, the research question and the explainability constraint **Project Stage:** Scientific framing

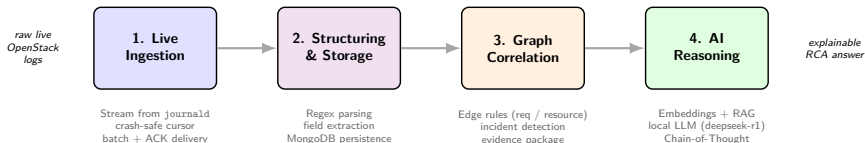
Central Research Question

OpenStack deployments split every user operation across many independent services, each producing its own logs, request IDs and timestamps. When a failure happens deep in that chain, how do we design a local-first system that continuously ingests raw journald evidence, reconstructs it as a correlation graph, and produces a bounded, explainable root-cause package – reasoned over by a fully local, open-weight LLM – without ever sending infrastructure data outside the organization's own machines?

Segmentation of the Problem

Role: Break the central question into four distinct engineering sub-problems

Project Stage: Scientific framing

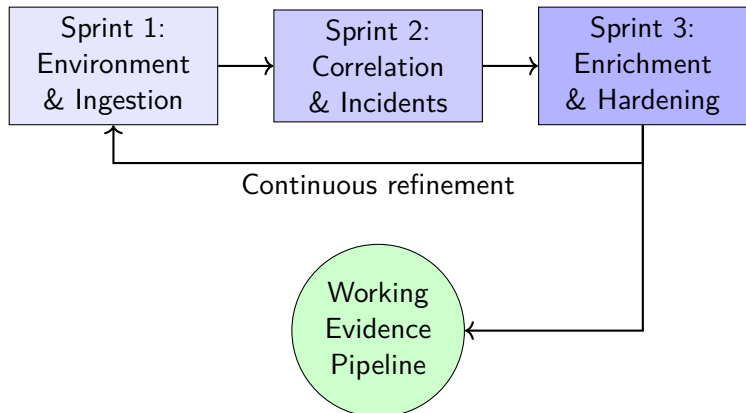


- **Stage 1:** logs never stop – ingestion must be live, resumable, and crash-safe.
- **Stage 2:** raw text is unusable – the parser extracts structured, queryable fields.
- **Stage 3:** “what led to what” is a graph problem, not a table filter.
- **Stage 4:** a local, open-weight model reasons over evidence – never raw logs.

Methodology: Agile Approach (Scrum)

Role: Show how the internship is split into work phases and iterations

Project Stage: Organization / planning



- **Flexibility:** adapting to real DevStack/OpenStack constraints as they appear.
- **Iteration:** each sprint hardens reliability before the next layer is added.

Project Management — Task Table

ID	Task Name	Duration	Predecessor(s) + link
A	Onboarding & problem framing (OpenStack, journald)	3 d	—
B	SOTA study: log-based RCA & streaming ingestion	4 d	A (0FD)
C	SOTA study: graph correlation & incident detection	4 d	A (0FD)
D	SOTA study: local LLM reasoning (CoT) & explainability	2 d	B (1DD), C (0FF)
E	Environment setup (DevStack, Tailscale, MongoDB)	5 d	A (0FD)
F	Ingestion & parsing pipeline (raw → parsed logs)	6 d	B (0FD), E (0FD)
G	Correlation engine (event_edges design)	7 d	C (0FD), E (0FD)
H	Incident integration on live correlated graph	5 d	F (0FD), G (0FD)
I	Enrichment worker (evidence package)	4 d	H (0FD)
J	Provider framework scaffolding	2 d	A (1DD)
K	Reliability hardening & unit tests	10 d	I (0FD), J (0FD)
L	Validation & end-to-end testing	3 d	K (0FD)
M	Final report, demo & presentation	5 d	L (0FD), D (0FD)

Notation

Example: 0FD means *Finish-to-Start, no lag*; 1DD means *Start-to-Start with a 1-day lag*. General notation: [lag] + [link type].

Project Management — Float Table

Role: Quantify the flexibility of each task through free and total float

Project Stage: Organization / planning

ID	A	B	C	D	E	F	G	H	I	J	K	L	M
FF	0	1	0	30	0	1	0	0	0	21	0	0	0
TF	0	2	1	30	0	1	0	0	0	21	0	0	0

FF = free float; **TF** = total float. Critical tasks verify **FF** = **TF** = 0, while **D** and **J** carry most of the schedule's slack.

Critical Path

A → E → G → H → I → K → L → M

Estimated total duration: 42 days (≈ 6-week internship)

Methodology — PERT Network Diagram

Role: Visualize the PERT network with earliest/latest dates and the critical path **Project Stage:** Organization / planning

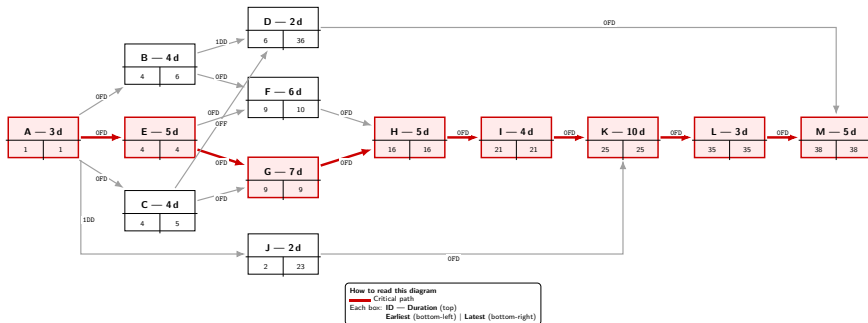


Figure 1: PERT network of the OpenStack RCA Copilot with critical path and earliest/latest dates.

Project Management — Gantt Chart

Role: Project the schedule onto a 6-week internship calendar respecting the DD, FD and FF links
Organization / planning

Project Stage:

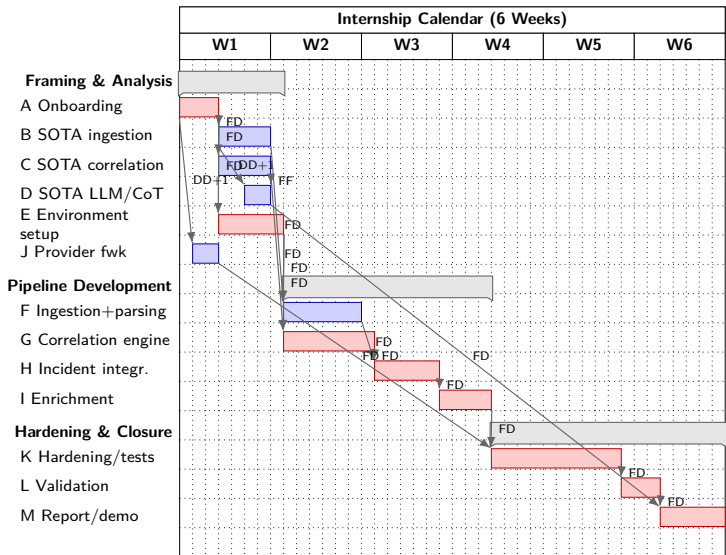


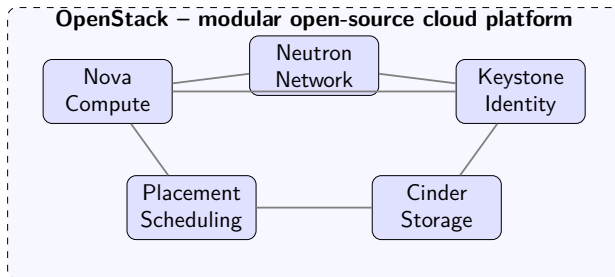
Table of Contents

- 1 Problem Statement and Methodology
- 2 OpenStack and the Debugging Challenge**
- 3 Data Ingestion and Structuring Pipeline
- 4 Correlation Graph and Evidence Package
- 5 AI Reasoning Layer
- 6 Data Sovereignty and Roadmap

OpenStack: A Powerful, Modular Cloud Platform

Role: Introduce the platform this project operates on

Project Stage: Technical context

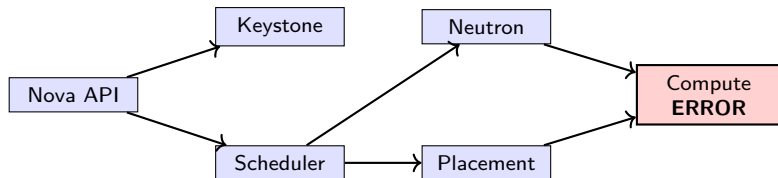


- Leading open-source IaaS platform, deployed at large scale worldwide.
- Modular by design: independent services for compute, network, identity, storage.
- Powers Huawei Cloud Stack's own private / sovereign cloud offering.

The Cost of Modularity: Hard-to-Backtrace Failures

Role: Motivate why root-cause analysis needs automated assistance

Project Stage: Technical context



same request_id shared across every hop

- Every strength above is also a debugging liability: one failure, many possible origins.
- A Nova error can trace back to Neutron, Keystone, Placement, or a stale event minutes earlier.
- Manually retracing this by hand does not scale – this is exactly where AI assistance earns its place.

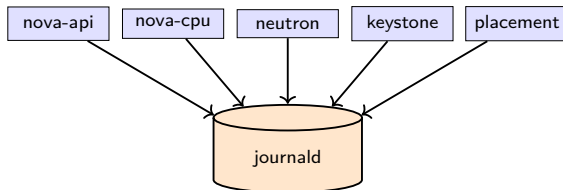
Table of Contents

- 1 Problem Statement and Methodology
- 2 OpenStack and the Debugging Challenge
- 3 Data Ingestion and Structuring Pipeline**
- 4 Correlation Graph and Evidence Package
- 5 AI Reasoning Layer
- 6 Data Sovereignty and Roadmap

Step 1: Why Logs Go to journald

Role: Explain why logs are centralized through journald

Project Stage: Pipeline – ingestion



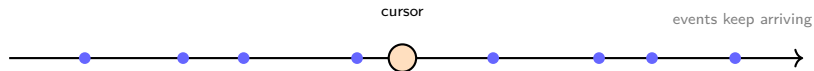
one structured, queryable stream –
not five scattered log files

- DevStack runs every OpenStack service as a systemd unit.
- journald gives ONE structured, queryable interface for all of them.
- No more hunting through separate files under `/var/log/nova`, `/var/log/neutron`...

Ingestion Is a Streaming Problem

Role: Explain why ingestion is a live streaming problem, not a batch read

Project Stage: Pipeline – ingestion



closed batch file



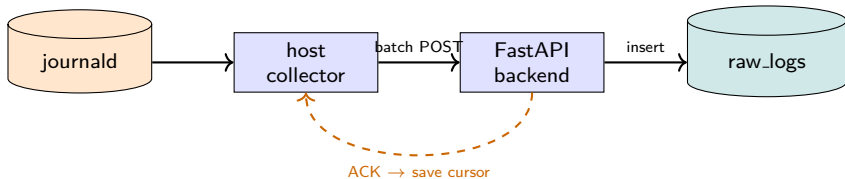
`journalctl -f` (live follow)

- Logs never stop while OpenStack runs – there is no “finished file” to read once.
- The collector follows the journal live, the same way you would `tail -f`.
- Every read is resumable via a cursor, or a crash loses the “where was I.”

From journald to MongoDB (raw_logs)

Role: Show how logs travel from journald to durable storage

Project Stage: Pipeline – ingestion

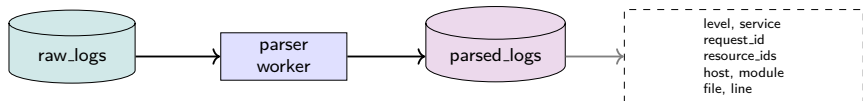


- Batches of about 50 records (or every 2 s) are POSTed to the backend.
- The cursor is saved only *after* a successful ACK – crash-safe delivery.
- Duplicates are caught by a (boot_id + journal_cursor) unique index.

Step 2: From Raw Text to Structured Events

Role: Show how raw text becomes structured, queryable events

Project Stage: Pipeline – structuring



- A separate parser worker reads raw_logs and writes parsed_logs – decoupled from ingestion.
- Extracts level, service, request_id, resource_ids, host, module, file, line.
- Parse failures are stored too (parse_status="failure"), never silently dropped.

Project Structure: Why This Layout

Role: Justify the repository layout and link to the source

Project Stage: Pipeline – structuring

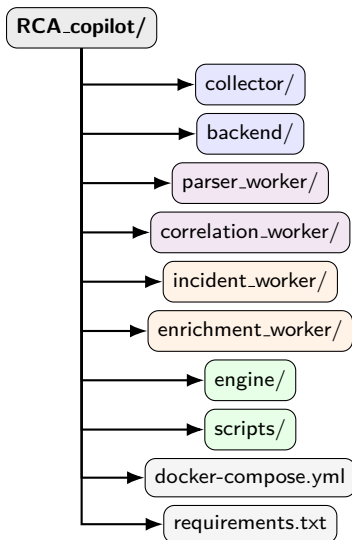


Figure 2: GitHub repository QR code

carteelttheboss/RCA_copilot – MIT Licensed

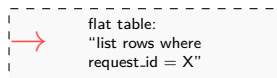
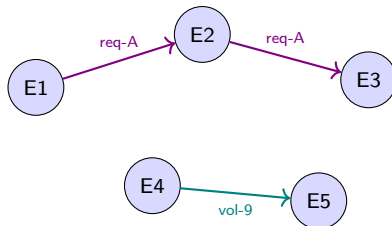
Table of Contents

- 1 Problem Statement and Methodology
- 2 OpenStack and the Debugging Challenge
- 3 Data Ingestion and Structuring Pipeline
- 4 Correlation Graph and Evidence Package**
- 5 AI Reasoning Layer
- 6 Data Sovereignty and Roadmap

Why This Is a Graph Problem

Role: Justify why correlation is modeled as a graph, not a table

Project Stage: Correlation – design



cannot express: $E1 \rightarrow E2 \rightarrow E3$

- Root-cause investigation asks “what led to what” – a chain question, not a filter question.
- Shared request_id / resource_id are natural EDGES between event NODES.
- A flat table can list matches; only a graph expresses a multi-hop evidence chain.

Building Correlation Edges Correctly

Role: Define the two deterministic correlation rules

Project Stage: Correlation – design



same_request_id
confidence 1.0 – 5 min window

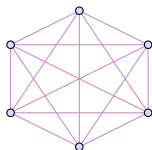
shared_resource_id
confidence 0.9 – 10 min window

- Two deterministic rules only – no black-box guessing.
- Every edge carries a reason and a confidence score.
- Edges are directed and chronological (earlier $\rightarrow$ later) – correlation, never proof of cause.

Avoiding the $O(n^2)$ Explosion

Role: Show the engineering fix that keeps the graph readable

Project Stage: Correlation – design



naive: $n(n-1)/2$ edges

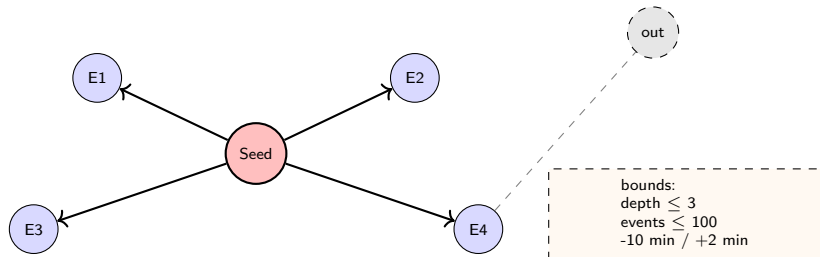
implemented: $n-1$ consecutive edges

- Naively linking every pair inside a shared-ID group explodes combinatorially.
- Fix: sort by time, link consecutive events only – skip periodic/noisy groups entirely.
- Same coverage, a fraction of the edges – a graph a human can actually read.

From Seed Event to Bounded Incident

Role: Show how a seed event becomes a bounded incident

Project Stage: Correlation – incidents

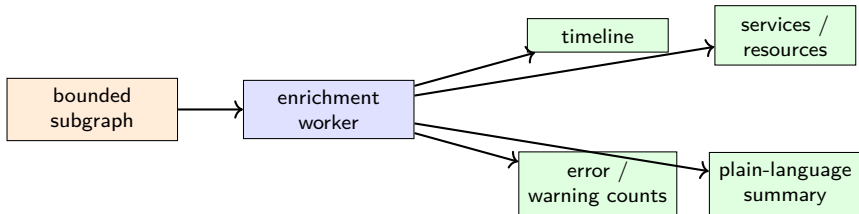


- Seed rules: ERROR/CRITICAL, Traceback, Exception, failed/timeout, resource ERROR state.
- Obvious false positives (e.g. "0 failures") are suppressed automatically.
- Bounded traversal keeps every incident small, readable, and explainable.

The Deterministic Evidence Package

Role: Show the deterministic package the AI layer will consume

Project Stage: Correlation – evidence



- Built entirely from stored events and edges – zero AI involved at this stage.
- Ordered timeline, impacted services/resources, error/warning counts, summary.
- This package – not raw logs – is what the LLM will eventually reason over.

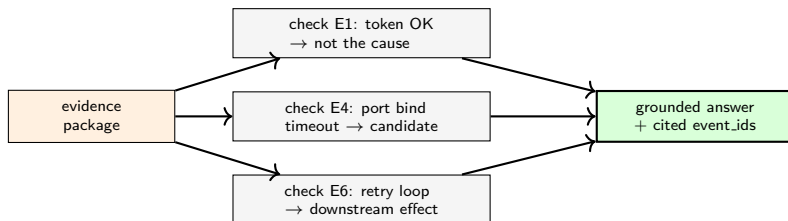
Table of Contents

- 1 Problem Statement and Methodology
- 2 OpenStack and the Debugging Challenge
- 3 Data Ingestion and Structuring Pipeline
- 4 Correlation Graph and Evidence Package
- 5 AI Reasoning Layer**
- 6 Data Sovereignty and Roadmap

Why Root-Cause Reasoning Needs Chain-of-Thought

Role: Explain why root-cause reasoning benefits from Chain-of-Thought

Project Stage: AI layer – reasoning

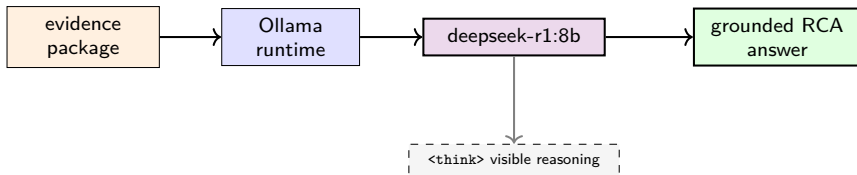


- A single stacked failure often has several plausible explanations.
- Chain-of-Thought: the model checks hypotheses one by one, out loud, before concluding.
- Every claim must cite a real event_id – no free-floating guesses.

The Model: Ollama + DeepSeek-R1

Role: Introduce the local reasoning model used for explanation

Project Stage: AI layer – model choice



- deepseek-r1:8b, served locally through Ollama – MIT-licensed, open weights.
- Reasoning is visible (<think> tokens) – transparent, auditable.
- A distilled reasoning model: near-frontier reasoning at laptop-GPU scale.

Prompt Engineering: Grounding the LLM in Evidence

Role: Show how the evidence package is turned into a grounded prompt

Project Stage: AI layer – prompting

Prompt = evidence package, not raw logs

```
SYSTEM: You are an OpenStack RCA assistant.  
Reason step by step over the evidence ONLY.  
Every claim must cite an event_id. If evidence  
is insufficient, say so explicitly.
```

```
USER: seed=E7 (ERROR, nova-compute)  
timeline=[E1..E7], edges=[...]  
Question: most likely root cause?
```

- The prompt IS the structured evidence package, not the raw logs.
- System prompt hard-requires a cited event_id for every claim.
- Same contract works behind any future provider (Ollama today, others later).

Why DeepSeek-R1?

Role: Explain the rationale behind selecting DeepSeek-R1

Project Stage: AI layer – model choice

- State-of-the-Art Performance.
- Open Weights.
- Data Sovereignty.

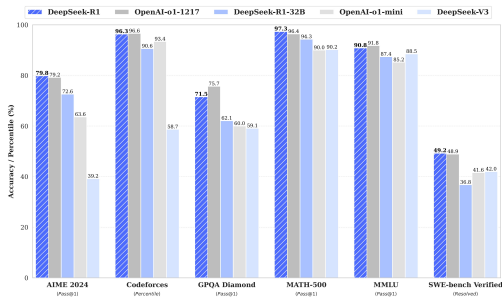
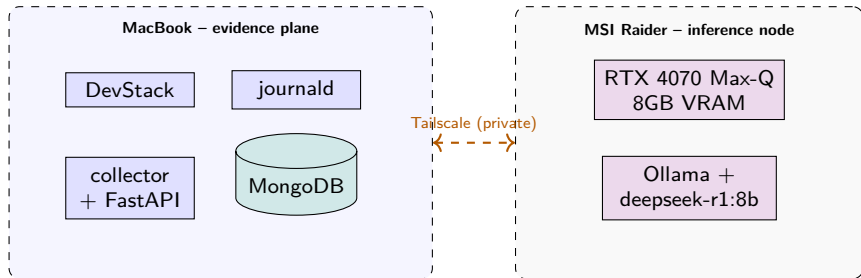


Figure 3: Official benchmark comparison (DeepSeek vs OpenAI o1)

Hardware: Two Machines, One Pipeline

Role: Describe the physical machines the pipeline runs on

Project Stage: AI layer – infrastructure



- MacBook: DevStack, journald, collector, FastAPI, MongoDB – the evidence plane.
- MSI Raider (RTX 4070 Max-Q, 8GB VRAM): local inference node.
- Linked privately over Tailscale – no public exposure, no cloud hop.

Why DeepSeek-R1 Fits This Hardware

Role: Justify the model choice against the available VRAM

Project Stage: AI layer – infrastructure

RTX 4070 Max-Q – 8GB VRAM budget



a 32B/70B model would overflow this GPU

- **Size calculation:** $8\text{B params} \times \frac{4\text{ bits}}{8} = 4\text{ GB (weights)}$ + $\sim 1\text{ GB (KV cache + runtime + Context Window)} \approx \mathbf{6.6\text{ GB}} \subset 8\text{ GB VRAM} \checkmark$
- Quantized deepseek-r1:8b fits comfortably inside 8GB of VRAM with $\sim 3\text{GB}$ headroom.
- Retains step-by-step reasoning distilled from the full R1 model.
- Right-sized: bigger models overflow this GPU, smaller ones reason worse.

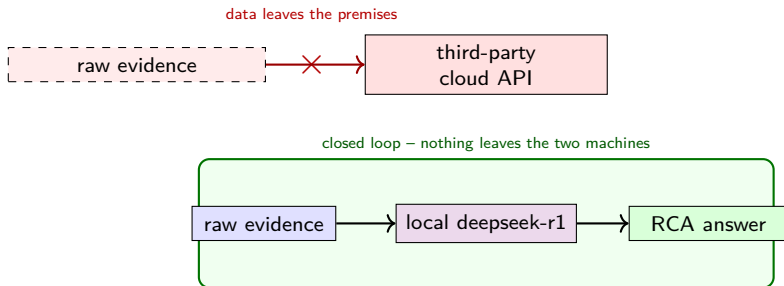
Table of Contents

- 1 Problem Statement and Methodology
- 2 OpenStack and the Debugging Challenge
- 3 Data Ingestion and Structuring Pipeline
- 4 Correlation Graph and Evidence Package
- 5 AI Reasoning Layer
- 6 Data Sovereignty and Roadmap**

Data Protection & Sovereignty

Role: State the core data-protection principle of the whole design

Project Stage: Data sovereignty



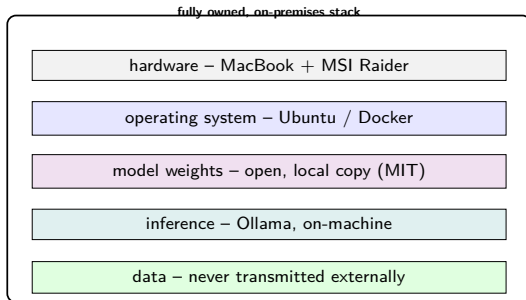
The Core Principle

A local, open-weight model running entirely offline on our own machines is the only way to get full control over both the data *and* the model itself.

The Sovereign AI Stack

Role: Show every layer of the stack under direct control

Project Stage: Data sovereignty



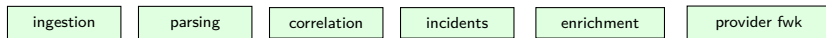
- Every layer of the stack is under direct organizational control.
- No vendor lock-in, no per-token cost, no data-residency ambiguity.
- Compliance by construction – not by contract or by trust.

Current Status: Proof of Concept

Role: Position what is delivered today against what comes next

Project Stage: Closure – PoC status

Implemented (PoC)



Next (Roadmap)



- This is a **PoC**: ingest → parse → correlate → detect → enrich is implemented and tested end to end.
- Provider framework (config, security, activation) is already built – ready to plug in Ollama.
- Next: wire live deepseek-r1:8b generation, then retrieval, then the Horizon UI.

Conclusion

Role: Close on the core value proposition

Project Stage: Closure

Key Takeaway

The OpenStack RCA Copilot turns scattered, cross-service logs into a bounded, explainable incident graph – reasoned over by a model that never leaves the building. The evidence pipeline is implemented and tested; the AI layer is the next milestone.

Thank you — Questions?